

Hybrid IDS - DNN & Logistic Regression

Deep Neural Network (DNN)

What is a DNN?

A Deep Neural Network is a machine learning model that processes data through multiple layers of connected nodes, loosely inspired by how neurons work in the brain. The "deep" part just means it has several hidden layers stacked between the input and output, each layer learns progressively more abstract patterns from the data.

In simple terms, you feed it raw features, and it figures out on its own which combinations of those features are useful for prediction. With enough data and the right architecture, it gets really good at picking up non-linear, complex patterns that simpler models would completely miss.

Role of DNN in the IDS

In this intrusion detection system, the DNN's job is to look at network flow features and classify each flow as either normal (BENIGN) or one of 11 known attack types. Network traffic can have very subtle statistical differences between attack types, so a deep model is well-suited here, it can learn those fine-grained distinctions that simpler classifiers might lump together.

In this project the DNN is specifically one of three base classifiers (alongside Random Forest and XGBoost). Its softmax probability outputs are then passed to a Logistic Regression meta-model which makes the final call. So the DNN doesn't work alone, it contributes its "vote" to a stacking ensemble.

Attacks the DNN is trained to detect:

- BENIGN : normal, legitimate network traffic (not an attack)
- Bot : automated bot traffic that manipulates or floods the network
- DDoS : distributed denial-of-service flooding from multiple sources
- DoS GoldenEye : HTTP-based DoS attack using the GoldenEye tool
- DoS Hulk : high-volume HTTP request flooding via the Hulk tool
- DoS Slowhttptest : slow HTTP attack targeting web server resources
- DoS slowloris : slow-rate connection exhaustion attack on HTTP servers
- FTP-Patator : brute-force password guessing on FTP services
- PortScan : scanning for open ports to discover network services
- SSH-Patator : brute-force password guessing on SSH services
- Web Attack | Brute Force : repeated credential stuffing on web login pages
- Web Attack | XSS : cross-site scripting injection via web requests

DNN Architecture

The model is a sequential feedforward network with 5 dense layers, batch normalization after the first three, and dropout throughout. Here's the full structure:

Layer	Type / Units	Details
Input	25 features	Normalized with StandardScaler beforehand
Dense 1	512 neurons (ReLU)	Learns broad feature combinations
BatchNorm + Dropout 35%	—	Stabilizes training, regularizes
Dense 2	256 neurons (ReLU)	More refined pattern detection
BatchNorm + Dropout 30%	—	Continues regularization
Dense 3	128 neurons (ReLU)	Compresses to key patterns
BatchNorm + Dropout 25%	—	Lighter regularization at this depth
Dense 4	64 neurons (ReLU)	Final feature extraction layer
Dropout 20%	—	Light regularization
Output	12 neurons (Softmax)	One probability per attack class

Total parameters: ~190,156 (about 188K trainable).

Key design decisions made:

- Batch Normalization after the wider layers (512, 256, 128) keeps gradients stable and lets the model train faster with less sensitivity to learning rate.
- Dropout rates decrease as the network narrows (35% → 20%) , more regularization where there's more capacity to overfit.
- Adam optimizer starting at lr=0.001 with ReduceLROnPlateau halving the rate every time val_loss plateaus for 4 epochs.
- EarlyStopping (patience=10) watched val_loss and stopped training at epoch 29, restoring weights from epoch 19 which had the best validation.
- Class weights set to balanced, even though training data was already SMOTE-oversampled to 290,700 samples per class — this keeps gradient updates fair.

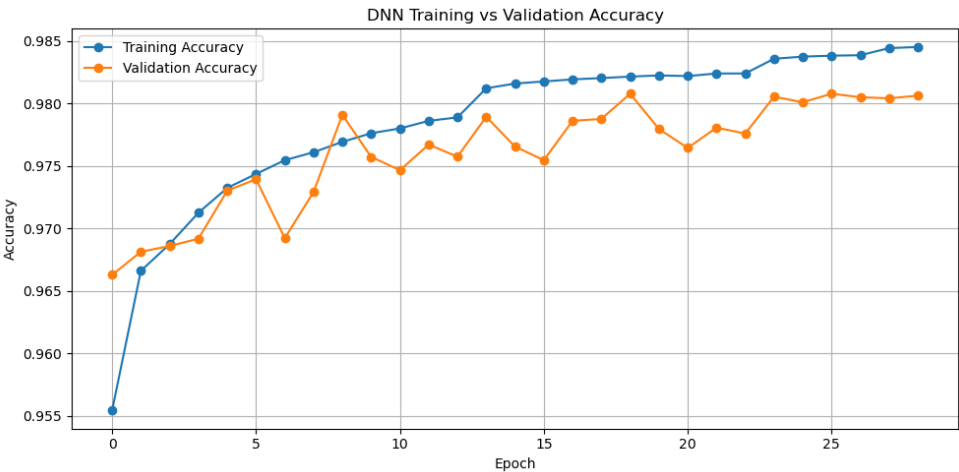


Figure 1: Training vs Validation Accuracy (29 epochs)

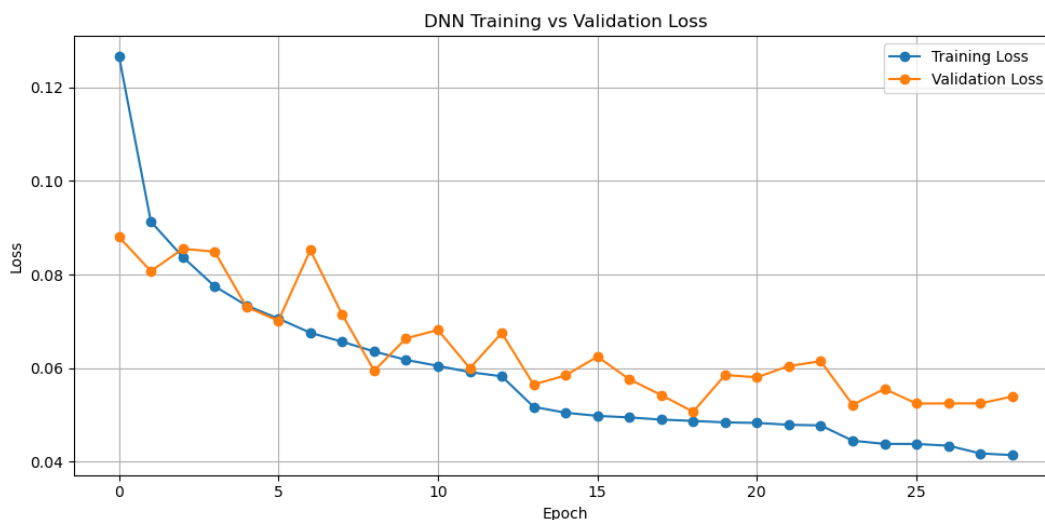


Figure 2: Training vs Validation Loss (29 epochs)

Evaluation Metrics

Overall Performance on Test Set

After early stopping at epoch 29 (best weights from epoch 19), the model was evaluated on 113,095 test samples:

Metric	Score
Accuracy	96.32%
Precision (weighted)	97.17%
Recall (weighted)	96.32%
F1-Score (weighted)	96.56%

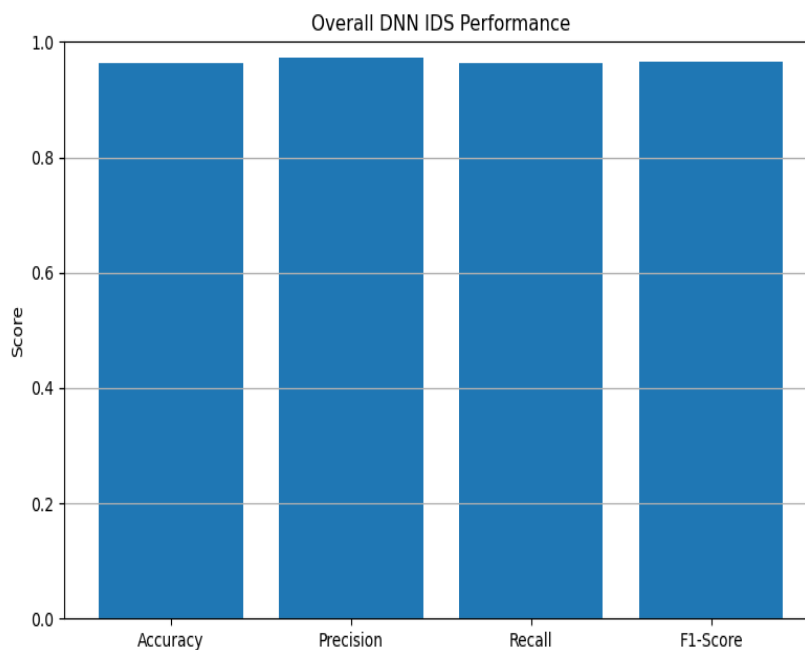


Figure 3: Overall DNN IDS Performance

Per-Class Performance Breakdown

Attack Type	Precision	Recall	F1-Score	Test Samples
BENIGN	0.999	0.954	0.976	90,844
Bot	0.313	0.972	0.473	74
DDoS	0.912	0.998	0.953	5,149
DoS GoldenEye	0.858	0.997	0.923	403
DoS Hulk	0.870	0.999	0.930	9,194
DoS Slowhttptest	0.820	0.986	0.895	218
DoS slowloris	0.823	0.986	0.897	223
FTP-Patator	0.753	1.000	0.859	315
PortScan	0.826	0.999	0.905	6,347
SSH-Patator	0.546	1.000	0.707	239
Web Attack Brute Force	0.179	0.733	0.288	60
Web Attack XSS	0.156	0.896	0.266	29

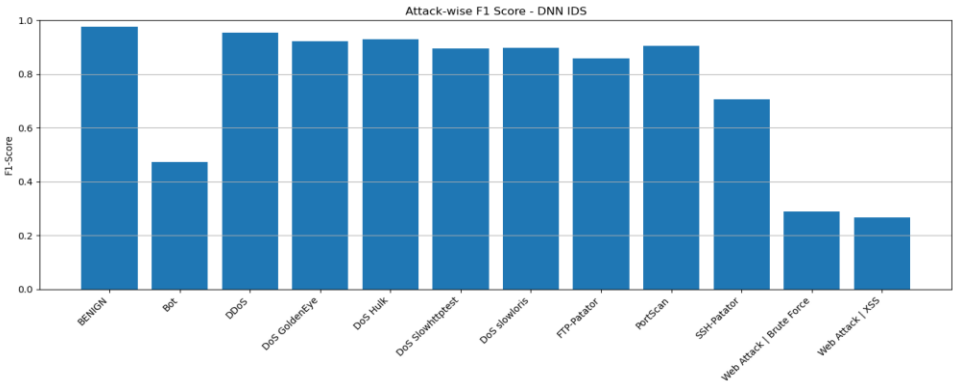


Figure 4: F1-Score per Attack Class

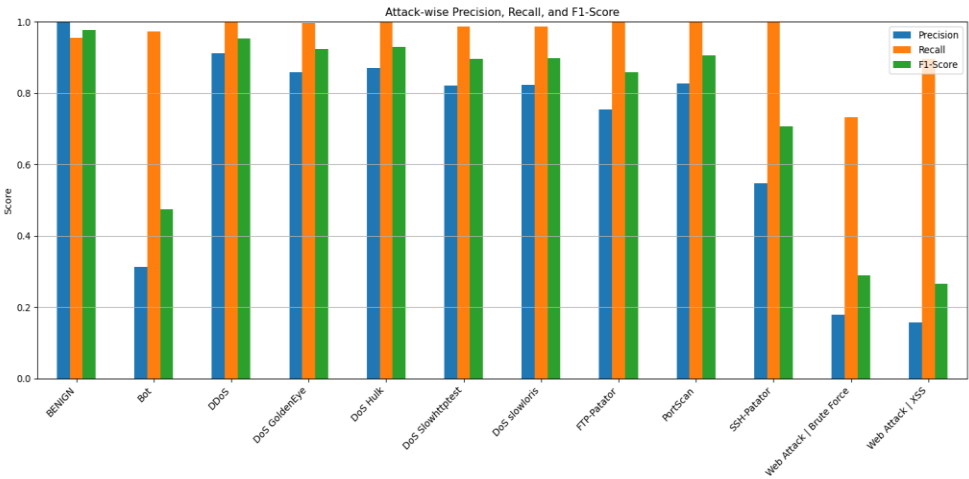


Figure 5: Precision, Recall, and F1-Score per Attack Class

Confusion Matrix

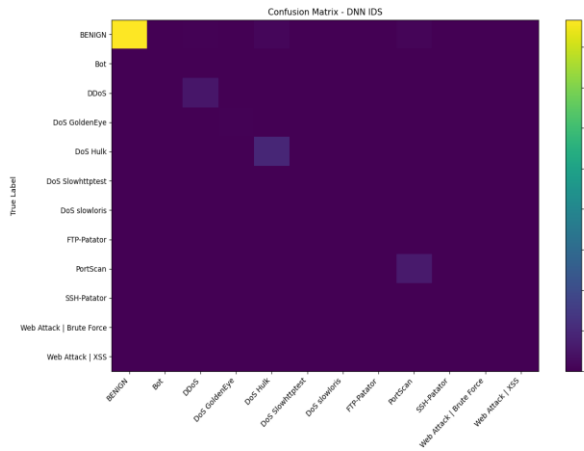


Figure 6: Raw Confusion Matrix

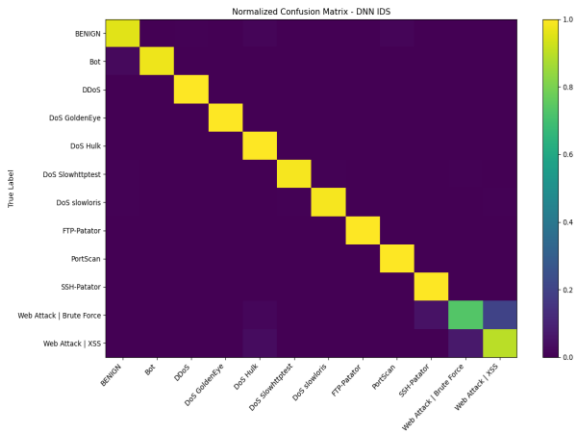


Figure 7: Normalized Confusion Matrix

The confusion matrix shows a very clean diagonal for most attack classes, DoS Hulk, PortScan, FTP-Patator, SSH-Patator all detected with near-perfect recall. The main issue is BENIGN being misclassified into various attack categories (4,128 false positives total):

- 1,370 BENIGN flows misclassified as DoS Hulk
- 1,328 misclassified as PortScan
- 494 misclassified as DDoS
- 198 as Web Attack | Brute Force
- 195 as SSH-Patator

ROC Curves

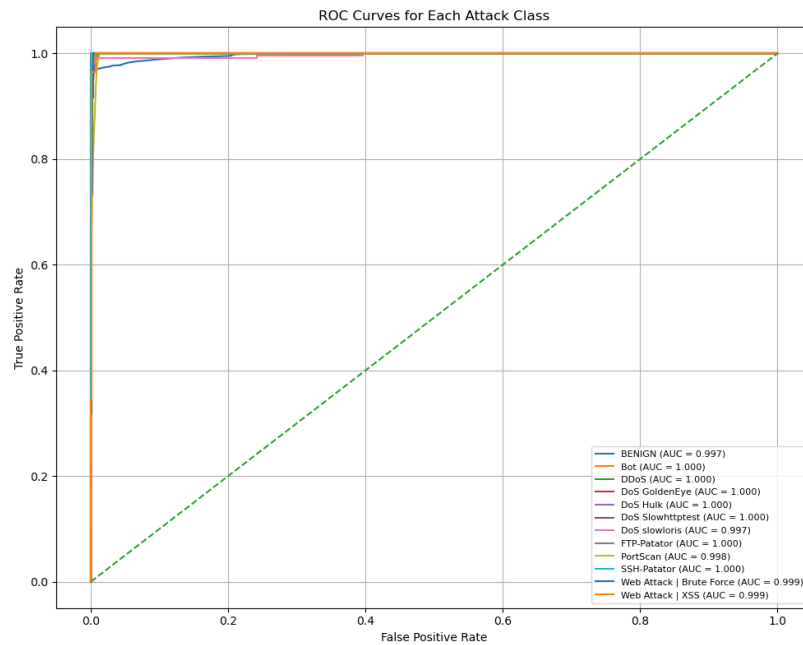


Figure 8: ROC Curves - AUC per Attack Class

The ROC curves confirm that the model has strong discriminative ability for most classes.

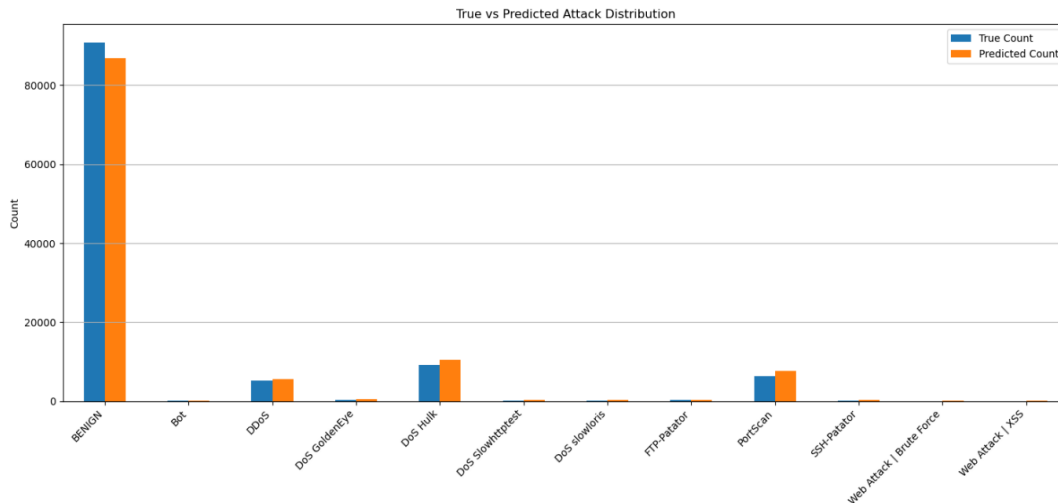


Figure 9: True vs Predicted Class Distribution

Attack-by-Attack Analysis

Strong performers (F1 > 0.90):

- **DDoS** (F1=0.954) : DDoS traffic has distinctive high-volume, low-variation flow patterns. Very easy for the network to spot once it's seen enough examples.
- **DoS GoldenEye** (0.923) & **DoS Hulk** (0.930) : Application-layer DoS creates characteristic HTTP request patterns. DNN picks these up cleanly.
- **DoS Slowhttptest** & **slowloris** (0.896, 0.898) : Slow attacks have abnormally long connection durations and low byte rates. Clear signals.
- **PortScan** (0.905) : Scanning traffic hits many destination ports with minimal data per connection. Very distinctive signature.

Decent but imperfect (F1 0.70–0.90):

- **FTP-Patator** (0.859) : Repetitive login attempts are clear, but some look enough like normal FTP to cause false positives.
- **SSH-Patator** (0.707) : SSH brute-force is harder since encrypted traffic looks similar at the flow level. Recall is perfect (100%) but precision suffers.
- **Bot** (0.474) : Very few test samples (74). The model catches nearly all of them (recall 0.973) but generates many false Bot alerts, dragging precision to 0.313.

Difficult attacks (F1 < 0.30):

- **Web Attack | Brute Force** (0.289) : Only 60 test samples and the model is over-sensitive, creating many false positives. Precision 0.18 means 82% of Brute Force predictions are actually wrong.
- **Web Attack | XSS** (0.267) : Worst performer. Only 29 test samples. XSS attacks embed malicious scripts in HTTP parameters, but at the network flow level, this looks almost identical to normal web browsing. Flow features like packet size, duration, and byte counts don't distinguish it.

Advantages of DNN in IDS

- Automatic feature learning : no need to hand-pick which features matter. The model discovers useful combinations on its own across all 25 input features.
- Handles non-linear patterns : attack behavior rarely maps to simple linear rules. Multiple layers let the DNN capture complex, curved decision boundaries.
- Scales to massive data : this model trained on 3.5 million samples. Traditional models like SVM would be impractical at this scale.
- High recall on most attack types : with class balancing, the DNN is very good at not missing attacks. That's generally what you want in security.
- Multi-class in one shot : a single model handles 12 categories simultaneously, versus training 12 binary classifiers.
- Softmax outputs are useful : the probability scores per class are informative and feed directly into the Logistic Regression meta-model. A model that just outputs 0/1 would lose this nuance.
- Generalizes across very different attack patterns from volumetric DDoS to slow-rate DoS to login brute force, all in one network.

Limitations

- Poor precision on rare classes: Web XSS and Brute Force suffer because the model doesn't have enough examples to learn precise boundaries. In production, constant false alarms would fatigue analysts.
- Black box : you can't easily explain why a specific flow was flagged. Security teams often need interpretable reasons, especially for incident response.
- Needs re-training for new attacks , unlike the Autoencoder in the ensemble, the DNN can only classify what it was trained on. A completely new attack type would get misclassified.

DNN Training Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
import tensorflow as tf

from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, roc_curve, auc
)
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.utils import to_categorical
```

```

np.random.seed(42)
tf.random.set_seed(42)

# Load preprocessed data
X_train = joblib.load("X_train.pkl")
X_test = joblib.load("X_test.pkl")
y_train = joblib.load("y_train.pkl")
y_test = joblib.load("y_test.pkl")

# Attack label mapping
attack_label_map = {
    0: "BENIGN", 1: "Bot", 2: "DDoS", 3: "DoS GoldenEye",
    4: "DoS Hulk", 5: "DoS Slowhttptest", 6: "DoS slowloris",
    7: "FTP-Patator", 10: "PortScan", 11: "SSH-Patator",
    12: "Web Attack | Brute Force", 14: "Web Attack | XSS"
}

original_classes = sorted(np.unique(y_train))
class_to_index = {cls: i for i, cls in enumerate(original_classes)}
index_to_class = {i: cls for cls, i in class_to_index.items()}

y_train = np.array([class_to_index[y] for y in y_train])
y_test = np.array([class_to_index[y] for y in y_test])

class_names = [attack_label_map[cls] for cls in original_classes]
num_classes = len(class_names)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train/val split
X_train_final, X_val, y_train_final, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

# One-hot encode
y_train_cat = to_categorical(y_train_final, num_classes=num_classes)
y_val_cat = to_categorical(y_val, num_classes=num_classes)
y_test_cat = to_categorical(y_test, num_classes=num_classes)

# Class weights
weights = compute_class_weight("balanced", classes=np.unique(y_train_final), y=y_train_final)
class_weights = dict(zip(np.unique(y_train_final), weights))

# Build DNN
model = Sequential([

```



```

    Input(shape=(X_train.shape[1],)),
    Dense(512, activation="relu"),
    BatchNormalization(),
    Dropout(0.35),
    Dense(256, activation="relu"),
    BatchNormalization(),
    Dropout(0.30),
    Dense(128, activation="relu"),
    BatchNormalization(),
    Dropout(0.25),
    Dense(64, activation="relu"),
    Dropout(0.20),
    Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Callbacks
early_stop = EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4, min_lr=1e-6)

# Train
history = model.fit(
    X_train_final, y_train_cat,
    validation_data=(X_val, y_val_cat),
    epochs=60, batch_size=256,
    class_weight=class_weights,
    callbacks=[early_stop, reduce_lr]
)

# Evaluate
y_proba = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_proba, axis=1)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="weighted", zero_division=0)
rec = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

print(f"Accuracy : {acc:.6f}")
print(f"Precision: {prec:.6f}")
print(f"Recall : {rec:.6f}")
print(f"F1 Score : {f1:.6f}")

```

```
print(classification_report(y_test, y_pred, target_names=class_names, zero_division=0))
```

DNN Conclusion

- 96.3% overall accuracy on 113,095 test samples across 12 classes.
- All DoS and volumetric attack types achieved $F1 > 0.89$, showing the DNN handles the bulk of common attacks well.
- Web-based attacks (XSS, Brute Force) remain weak.
- In the full ensemble, RF and XGBoost may compensate for DNN's weak spots on web attacks, and vice versa for complex volumetric patterns.

Logistic Regression as the Meta-Model

What is Logistic Regression?

Logistic Regression is a classification algorithm that models the probability of each class using a sigmoid (binary) or softmax (multi-class) function applied to a linear combination of the input features. Despite the name, it's a classifier, it predicts categories.

As a meta-model in a stacking ensemble, it doesn't see raw network traffic at all. Instead, it sees the predicted probabilities from the three base classifiers plus the Autoencoder reconstruction error, and learns to combine those signals into a final decision.

Role in of Logistic Regression in IDS

The feature fusion step concatenates:

- 12 probability values from Random Forest (one per class)
- 12 probability values from XGBoost
- 12 probability values from DNN (softmax output)
- 1 reconstruction error (MSE) from the Autoencoder

That gives Logistic Regression a 37-dimensional input for each sample. It then learns which combinations of these probability signals most reliably predict the final class. In effect, LR learns to arbitrate between the base models, it might trust XGBoost more for PortScan but rely on DNN more for DoS attacks.

Why Logistic Regression for the meta-level?

- it's fast to train (the hard computation is already done)
- it's interpretable (you can inspect coefficients to see which base model it trusts more for which class)
- it doesn't easily overfit with only 37 input features. It acts like a learned, adaptive voting mechanism.

Model Comparison in the Hybrid IDS

Model	Type	Role	Strength	Weakness
Random Forest	Tree ensemble	Base classifier	Robust to noise, handles mixed feature types, feature importance is interpretable	Slower on very large data, doesn't capture complex sequential patterns
XGBoost	Gradient boosting	Base classifier	Generally best tabular-data performance out of the box, handles imbalance well with scale_pos_weight	Can overfit without tuning, many hyperparameters to manage
DNN	Neural network	Base classifier	Learns complex non-linear patterns automatically, scales well, outputs calibrated probabilities	Black box, slow to train on CPU, needs many samples for rare classes
Autoencoder	Neural network	Anomaly detector	Can detect unseen/zero-day attacks via high reconstruction error — doesn't need attack labels	Only detects anomalies, can't classify which attack type it is
Logistic Regression	Linear classifier	Meta-model (stacker)	Fast, interpretable, robust combiner of base model outputs, unlikely to overfit at meta level	Can't capture non-linear relationships between base model outputs

CODE

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
import tensorflow as tf

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.preprocessing import label_binarize
from tensorflow.keras.models import load_model

np.random.seed(42)
tf.random.set_seed(42)

print("=" * 80)
print("HYBRID IDS - LOGISTIC REGRESSION META MODEL")
print("=" * 80)

```

```

print("\nLoading data...")
X_train = joblib.load("X_train.pkl")
X_test = joblib.load("X_test.pkl")
y_train = joblib.load("y_train.pkl")
y_test = joblib.load("y_test.pkl")

print(f"Original X_train shape: {X_train.shape}")
print(f"Original X_test shape : {X_test.shape}")

class_to_index = joblib.load("class_to_index.pkl")
index_to_class = joblib.load("index_to_class.pkl")
class_names = joblib.load("class_names.pkl")

original_classes = sorted(class_to_index.keys())
num_classes = len(class_names)

y_train = np.array([class_to_index[y] for y in y_train])
y_test = np.array([class_to_index[y] for y in y_test])

print(f"\nDetected attack classes ({num_classes}):")
for i, cls in enumerate(class_names):
    print(f"{i} -> {cls}")

print("\nLoading scaler...")
scaler = joblib.load("scaler.pkl")

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nLoading trained base models...")

rf_model = joblib.load("random_forest_model.pkl")
xgb_model = joblib.load("xgboost_model.pkl")
dnn_model = load_model("dnn_ids_model.keras")
autoencoder_model = load_model("autoencoder_model.keras")

print("All models loaded successfully.")

print("\nGenerating Random Forest probabilities...")
rf_train_proba = rf_model.predict_proba(X_train)
rf_test_proba = rf_model.predict_proba(X_test)

print("Generating XGBoost probabilities...")
xgb_train_proba = xgb_model.predict_proba(X_train)
xgb_test_proba = xgb_model.predict_proba(X_test)

print("Generating DNN probabilities...")
dnn_train_proba = dnn_model.predict(X_train_scaled, verbose=0)

```

```

dnn_test_proba = dnn_model.predict(X_test_scaled, verbose=0)

print("Generating Autoencoder reconstruction error...")
ae_train_recon = autoencoder_model.predict(X_train_scaled, verbose=0)
ae_test_recon = autoencoder_model.predict(X_test_scaled, verbose=0)

ae_train_error = np.mean(np.square(X_train_scaled - ae_train_recon), axis=1).reshape(-1, 1)
ae_test_error = np.mean(np.square(X_test_scaled - ae_test_recon), axis=1).reshape(-1, 1)

print("\nCreating stacked meta-features...")
X_meta_train = np.hstack([
    rf_train_proba,
    xgb_train_proba,
    dnn_train_proba,
    ae_train_error
])

X_meta_test = np.hstack([
    rf_test_proba,
    xgb_test_proba,
    dnn_test_proba,
    ae_test_error
])

print(f"Meta-train shape: {X_meta_train.shape}")
print(f"Meta-test shape : {X_meta_test.shape}")

print("\nTraining Logistic Regression meta-model...")
meta_model = LogisticRegression(
    max_iter=2000,
    multi_class="multinomial",
    solver="lbfgs",
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

meta_model.fit(X_meta_train, y_train)
print("Meta-model training completed.")

print("\nGenerating final hybrid predictions...")
y_proba = meta_model.predict_proba(X_meta_test)
y_pred = np.argmax(y_proba, axis=1)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="weighted", zero_division=0)
rec = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

```

```
print("\n" + "=" * 80)
print("OVERALL HYBRID IDS PERFORMANCE")
print("=" * 80)
print(f"Accuracy : {acc:.6f}")
print(f"Precision: {prec:.6f}")
print(f"Recall : {rec:.6f}")
print(f"F1 Score : {f1:.6f}")
```

Conclusion

The Hybrid IDS gives better attack detection by combining the strengths of multiple models instead of depending on just one.

DNN helps in learning complex attack patterns, while Random Forest, XGBoost, and Autoencoder support better classification and anomaly detection.

Logistic Regression combines all their outputs and makes the final prediction more accurate and reliable.

Overall, this system is effective for detecting different types of network attacks and is suitable for real-world IDS applications.